

Stemz HR Assistant

Security & Robustness Hardening — Change Report

A detailed record of what changed, why it changed, and where — organised by priority. Architecture was left unchanged; the three-app RAG pipeline (knowledge → rag → chatbot) is identical.

Date	August 2026
Scope	High / Medium / Low priority fixes (no architecture change)
Validation	Django check clean · 14 new unit tests pass · 62 retrieval baseline tests pass

At a glance

Priority	Area	Files touched
High	Settings hardening	config/settings/base.py
High	CSRF protection restored	views.py, chat.html
High	Input validation & rate limiting	views.py
Medium	Thread-safe model caches	embedder.py, reranker.py
Medium	Atomic ingest writes	ingest.py
Medium	Real YAML frontmatter parsing	parser.py
Medium	LLM timeouts & filename escaping	generate.py, titler.py
Medium	Document extraction guard	document.py
Low	Config & dependency hygiene	.env.example, requirements.txt
Low	Unit tests & markers	tests/test_units.py, pytest.ini

High priority — security & correctness

1. ALLOWED_HOSTS no longer breaks production

Where: `config/settings/base.py`

Why & what: Previously `ALLOWED_HOSTS` was `['*']` if `DEBUG` else `[]`. The moment `DEBUG=False` (i.e. production), the list became empty and Django rejected **every** request with HTTP 400 — a self-inflicted outage. It now reads a comma-separated list of real hostnames from an environment variable in production, which is how Django is designed to be configured.

```
ALLOWED_HOSTS = (
    ['*'] if DEBUG
    else [h.strip() for h in os.getenv('DJANGO_ALLOWED_HOSTS', '').split(',') if
h.strip()]
)
```

2. Fail loudly on insecure SECRET_KEY in production

Where: `config/settings/base.py`

Why & what: The `SECRET_KEY` signs session cookies, CSRF tokens and password-reset tokens. If it silently fell back to the shipped `django-insecure-...` default, anyone reading the source could forge those. Django gives no warning — it just runs insecurely. We now raise `ImproperlyConfigured` at startup so the misconfiguration is impossible to miss. This is a no-op in dev (`DEBUG=True`).

```
if not DEBUG and SECRET_KEY == _INSECURE_SECRET_KEY:
    raise ImproperlyConfigured('DJANGO_SECRET_KEY must be set ... when DJANGO_DEB
UG=False.')
```

3. Reject the default admin password in production

Where: `config/settings/base.py`

Why & what: The `/dashboard/` route is protected by HTTP Basic Auth, but the password defaulted to `change-me`. A guessable default on an admin surface is effectively no authentication. The same startup guard now forces a real value in production.

```
if not DEBUG and ADMIN_PASSWORD in ('', 'change-me'):
    raise ImproperlyConfigured('ADMIN_PASSWORD must be set ... when DJANGO_DEBUG=
False.')
```

4. Production security flags (cookies, HSTS, SSL)

Where: `config/settings/base.py`

Why & what: Added Django's standard production hardening: secure-only cookies, HSTS to prevent protocol downgrade, and no MIME sniffing. Everything is gated behind `not DEBUG` so local http development is unaffected. The SSL redirect and HSTS max-age are env-overridable for setups that terminate TLS at a proxy.

```

if not DEBUG:
    SESSION_COOKIE_SECURE = True
    CSRF_COOKIE_SECURE = True
    SECURE_CONTENT_TYPE_NOSNIFF = True
    SECURE_SSL_REDIRECT = ...
    SECURE_HSTS_SECONDS = ...

```

5. CSRF protection restored on all endpoints

Where: `apps/chatbot/views.py` + `apps/chatbot/templates/chatbot/chat.html`

Why & what: This was the biggest gap. Every state-changing endpoint carried `@csrf_exempt`. Because the browser auto-attaches the session cookie to any request to your domain, a malicious page a signed-in user visits could silently POST to these endpoints as that user — deleting conversations or firing off Ask-HR emails. Session-key isolation does not help, because the victim's own cookie is what gets abused. The fix has three parts that must work together: (a) `@ensure_csrf_cookie` on the page view sets the `csrftoken` cookie; (b) all six `@csrf_exempt` decorators removed, re-enabling Django's CSRF middleware; (c) a `csrfHeaders()` helper in the frontend attaches the `X-CSRFToken` header to all seven mutating `fetch()` calls. Django validates the header against the cookie (double-submit), which a cross-site attacker cannot read.

```

// frontend helper
function csrfHeaders(extra) {
  return Object.assign({ 'X-CSRFToken': getCookie('csrftoken') }, extra || {});
}

```

6. Input validation & rate limiting

Where: `apps/chatbot/views.py`

Why & what: Three abuse vectors were open. (a) Chat messages were unbounded — a multi-megabyte paste would be embedded and sent to Claude, spiking cost and latency (denial-of-wallet); now capped at 5,000 characters. (b) The Ask-HR email was only stripped, never validated; it now passes through Django's `validate_email`. (c) Nothing throttled the endpoints — chat spam (LLM cost), transcribe spam (CPU-heavy Whisper) and Ask-HR spam (flooding HR). A lightweight per-session cache-backed rate limiter now guards them with generous limits (30/min chat, 20/min upload & transcribe, 5/min ask-hr). No new dependency was added.

```

@rate_limit('chat', limit=30, window_s=60)
...
if len(query) > MAX_QUERY_CHARS:      # 5000
    return JsonResponse({'error': 'Message too long...'}, status=400)

```

7. Stop leaking internal errors to clients

Where: `apps/chatbot/views.py` (`transcribe_audio`)

Why & what: The transcription handler returned the raw exception string to the browser, which can expose file paths, model names or internal state. The detail now stays server-side and the user receives a generic message.

```
except Exception:  
    return JsonResponse({'error': 'Transcription failed. Please try again.'}, sta  
tus=500)
```

Medium priority — robustness

8. Thread-safe model caches

Where: *apps/knowledge/services/embedder.py, apps/rag/services/reranker.py*

Why & what: The embedding and reranker models are cached in a module global and lazy-loaded on first use. Under a threaded server, two concurrent first-requests could each see the cache empty and both load the (large) model, doubling memory and racing on assignment. Added double-checked locking — a cheap check first, a lock only on the cold path, then a re-check inside the lock. This matches the pattern already used in *asr.py*.

```
if _MODEL is None:
    with _LOCK:
        if _MODEL is None:
            _MODEL = SentenceTransformer(MODEL_NAME)
```

9. Reranker degrades gracefully instead of 500-ing

Where: *apps/rag/services/reranker.py*

Why & what: If the cross-encoder failed (out-of-memory, corrupt weights) the whole request used to error out. It now falls back to the first-stage cosine ordering, so the user gets a slightly less-optimal answer rather than a failure.

```
try:
    scores = model.predict(pairs)
except Exception:
    return retrieved[:top_k]
```

10. Atomic ingest writes

Where: *apps/knowledge/services/ingest.py*

Why & what: Retrieval depends on *chunks.json* and *embeddings.npy* staying in lockstep — row *i* of the matrix must match chunk *i*. The old code wrote them sequentially in place, so a crash or disk-full between the two writes left them desynced, silently corrupting every future answer. We now write to temp files and *os.replace* them — an atomic OS-level swap, so either the whole old pair or the whole new pair is present, never a half-written mix. A file handle is passed to *np.save* so it doesn't append its own *.npy* to the temp name.

```
tmp_chunks.write_text(...)
with open(tmp_emb, 'wb') as f:
    np.save(f, embeddings_matrix)
os.replace(tmp_chunks, chunks_path)
os.replace(tmp_emb, emb_path)
```

11. Real YAML frontmatter parsing

Where: *apps/knowledge/services/parser.py*

Why & what: The old parser split each line on the first colon, which breaks on any value containing a colon (e.g. "see clause 4:30") or quoted strings, silently truncating metadata. It now uses `yaml.safe_load`, with the naive splitter kept as a fallback if PyYAML is missing. `safe_load` (never load) avoids YAML's arbitrary-object-construction risk on untrusted input.

```
loaded = yaml.safe_load(frontmatter_raw)
if isinstance(loaded, dict):
    return {str(k): ('' if v is None else str(v)) for k, v in loaded.items()}
```

12. LLM timeouts & filename escaping

Where: `apps/rag/services/generate.py`, `apps/chatbot/services/titler.py`

Why & what: No timeout meant a hung Anthropic request could pin a server worker indefinitely; enough of those exhausts the worker pool. Bounded waits (120s generate, 15s titler) let requests fail cleanly. Separately, the user-controlled uploaded filename was interpolated raw into an XML-ish tag the model reads — a filename with a quote or angle bracket could break the tag; it is now `html.escape`'d. The titler also now logs its fallback instead of swallowing the error silently (an invalid API key would otherwise mis-title every chat forever with no signal).

```
client = Anthropic(api_key=..., timeout=LLM_TIMEOUT_S)
...
f'<document filename="{html.escape(filename or "", quote=True)}">'
```

13. Document extraction guard

Where: `apps/chatbot/services/document.py`

Why & what: A file renamed to `.pdf` but containing junk (or a genuinely corrupt PDF) raised a low-level library exception that surfaced as an HTTP 500. Any extractor failure now becomes a clean, user-facing `DocumentError` (which the view already turns into a friendly 400). Our own intentional `DocumentErrors` are re-raised unchanged so their specific messages survive.

```
try:
    ... extract ...
except DocumentError:
    raise
except Exception:
    raise DocumentError('Could not read this file as ...')
```

LOW

Low priority — config, dependencies, tests

14. Complete .env.example

Where: `.env.example`

Why & what: It listed only 6 of the ~25 variables the settings actually read. Anyone copying it would boot with missing email/admin/whisper/model config and hit runtime failures. Every variable is now documented with its purpose, and which are required in production.

15. Pinned dependencies

Where: `requirements.txt`

Why & what: Added upper version bounds (e.g. `anthropic>=0.30.0,<1.0`) so a future breaking release cannot silently break a fresh install, and made `faster-whisper` and `PyYAML` explicit rather than assumed to be present.

16. Hermetic unit tests + markers

Where: `tests/test_units.py`, `pytest.ini`

Why & what: The only existing tests covered retrieval. Added 14 fast tests (no network, DB or models) for the pure logic that changed: frontmatter parsing including the colon-in-value case, guardrail decisions, document extraction error paths, and titler formatting. They run in ~4 seconds — cheap regression protection. `slow/integration` markers were added to `pytest.ini` for future organisation.

Intentionally left unchanged

`is_confidential` (guardrail) was *not* made fail-closed. The instinct is to treat a missing `confidential` field as confidential — but the parser already sets that field on **every** chunk, so a fail-closed default would misfire and block all normal, non-confidential policies. The current behaviour is correct given how chunks are built.

Validation: **Django system check** reported no issues; **14 new unit tests** pass; and the full **62-case retrieval baseline** still passes — confirming the reranker refactor is behaviour-preserving.